Support Ticket Triage with Google Gemini

Project overview | Source code: https://github.com/shay2222/support-triage | Live app: https://support-triage-qbzvsmzvf8smjdbuvykeqc.streamlit.app/

1. Purpose

Support Ticket Triage is a working LLM application that reads a customer message and returns three structured fields: a **category** (bug, billing, feature_request, or other), an **urgency** level (low, medium, or high), and a one-sentence **summary**. It is built on the Google Gemini Developer API and deployed as a public web app with source code on GitHub.

The project is intentionally narrow. Its purpose is to demonstrate a complete production path for one LLM feature: defining the problem, designing a prompt and output schema, validating and retrying model output, measuring accuracy against labeled examples, handling failures, and shipping an interface people can use. Each of these steps is a recurring engineering problem for any startup building on an LLM API.

2. How it helps startups

Early-stage companies often receive support requests through email, chat, forms, and social media, usually handled by one or two people who also do everything else. Three problems follow:

- **Slow response to urgent problems.** An outage report sits in the same queue as a feature suggestion. Urgent issues get noticed late, which damages trust with the customers who are most affected.
- **Inconsistent handling.** Different people categorize the same message differently, so bugs go to the wrong place and billing questions wait for the wrong person.
- **No data.** Without structured fields, a founder cannot see whether bugs or billing problems are rising, which slows product and pricing decisions.

The tool addresses all three with a first pass that takes seconds per message. Every output is structured, so it can feed a helpdesk, a Slack alert, a dashboard, or a routing rule. Because accuracy is measured against labeled examples, a founder can decide how much to trust it before automating anything.

The same design applies to other early-stage problems:

- **Lead routing:** classify inbound sales inquiries by fit and intent, and send hot leads to a founder immediately.
- **Bug intake:** convert free-text bug reports into engineering tickets with severity and component fields.
- **Feedback analysis:** tag product feedback by theme so a small team can see what customers ask for most.
- **Finance operations:** route invoices or expense claims to the right approver with a category and risk flag.

The key idea is that the model does not replace a person. It handles the repetitive first pass, and a human reviews the high-urgency and uncertain cases. This keeps the cost of a mistake low, which is how startups can adopt AI without risking customer relationships.

3. Technical design

The system has four layers: a prompt and schema layer, a model call layer with validation, an evaluation layer, and an interface layer. Each is a separate file, so the components can be changed or tested independently.

File	Layer	Responsibility
triage.py	Core logic	Defines categories and urgency levels, builds the prompt, calls the model with JSON output and temperature 0, validates every field, and retries on invalid output. Raises a typed TriageError when all attempts fail.
eval.py	Evaluation	Runs every labeled ticket through triage_ticket, records expected and predicted labels for each row, counts category and urgency accuracy, and writes results/eval_results.csv for error analysis. Paces requests to respect free-tier rate limits.
app.py	Interface	Streamlit page with a text area, a Triage button, metric cards for category and urgency, a summary, and an expandable raw JSON view. Reads the API key from environment variables locally or from Streamlit secrets when deployed.
data/sample_tickets.csv	Test data	20 hand-labeled support messages with expected category and urgency. The labels are the ground truth the evaluation measures against.
requirements.txt	Dependencies	google-genai and streamlit, pinned to the packages the app imports.
README.md	Documentation	Problem statement, setup steps, results with status, design decisions, and limitations.

3.1 Model call and output contract

The model is called through the official `google-genai` SDK using the `generate_content` method. Two configuration choices shape the output:

- `response_mime_type="application/json"` asks the model to return JSON, so the response can be parsed with `json.loads` instead of fragile text matching.
- `temperature=0` reduces sampling randomness, so the same message is more likely to get the same label on repeat runs. This matters for evaluation, because a score that changes on every run cannot be compared over time.

The prompt lists the allowed values for each field and includes a written urgency guide: high for outages, data loss, security concerns, or a customer who cannot work; medium for a problem with a workaround; low for questions and suggestions. Writing the rule down in the prompt makes the behavior explicit and easy to revise, which is more reliable than hoping the model infers the team's standard.

3.2 Validation and retry

A model can return valid JSON with a value outside the allowed list, or leave out a field. The `_validate` function checks that category and urgency belong to their enumerated sets and that the summary is a non-empty string. Any failure raises a `TriageError`, and the call is retried up to two more times. If every attempt fails, the error is raised to the caller rather than returning a guessed label. In practice this means the system either returns a schema-conforming result or reports a failure that can be logged and reviewed. It never passes a malformed label downstream silently.

3.3 Evaluation harness

Most demo projects show that a model can answer. This project measures how often it answers correctly. `eval.py` compares each prediction to a labeled expectation and records both fields separately, because category and urgency fail for different reasons. The output is per-ticket: a row shows the message, the expected and predicted labels, and whether each field matched. The results CSV is the starting point for error analysis, and it is kept out of version control so that private results are not published accidentally.

This structure is the basis for regression testing. When a prompt or model changes, the same 20 tickets can be rerun, and any change in accuracy is visible as a number rather than a feeling.

3.4 Error handling and operational concerns

- **Quota and rate limits:** the evaluation pauses between requests to stay within free-tier limits, and a quota error is reported with its message rather than hidden.
- **Model retirement:** the model name is one configuration constant. When Google retired the first model I used, the fix was a one-line change, which is the reason to keep model names in one place.
- **Secrets:** the API key is read from environment variables or Streamlit secrets. It is excluded from version control by .gitignore, and the app stops with a clear message if the key is missing.
- **Per-request cost:** each classification is one short request, which is why this design is cheap to run. Cost scales linearly with ticket volume, so a team can estimate spend from the daily ticket count.

3.5 Stack

- Python 3.10+ with the google-genai SDK
- Google Gemini Developer API (current model configured in triage.py)
- Streamlit for the interface, deployed on Streamlit Community Cloud from the GitHub repository
- GitHub for source control, with GitHub Codespaces used as the development environment

4. Results

Status: partial evaluation. The free-tier daily request quota was reached during the full 20-ticket run. The numbers below are reported exactly as measured so far, and the README states this clearly. The full run will be repeated when the quota resets, and the results will be updated then.

Metric	Result so far
Category accuracy	13 of 13 tickets evaluated (100%)
Urgency accuracy	11 of 13 tickets evaluated (85%)
API errors	0

4.1 Urgency misses and analysis

- **Ticket 2** (a double charge on a subscription, labeled high): a billing error is serious, but it does not block use of the product. Under the written guide, medium may be the more defensible label. This is a labeling ambiguity rather than a clear model failure.

- **Ticket 10** (an account cancellation with a receipt request, labeled medium): nothing is broken and no workaround is needed, so low may fit better under the current guide. The model's answer may be the more accurate one.

Both misses show that urgency is hard to define consistently. They lead to a specific next step: review the labels, make the urgency rules more precise, and record the before-and-after numbers. Changing the prompt to fit the 20 examples would overfit, so any change should be tested on a larger set.

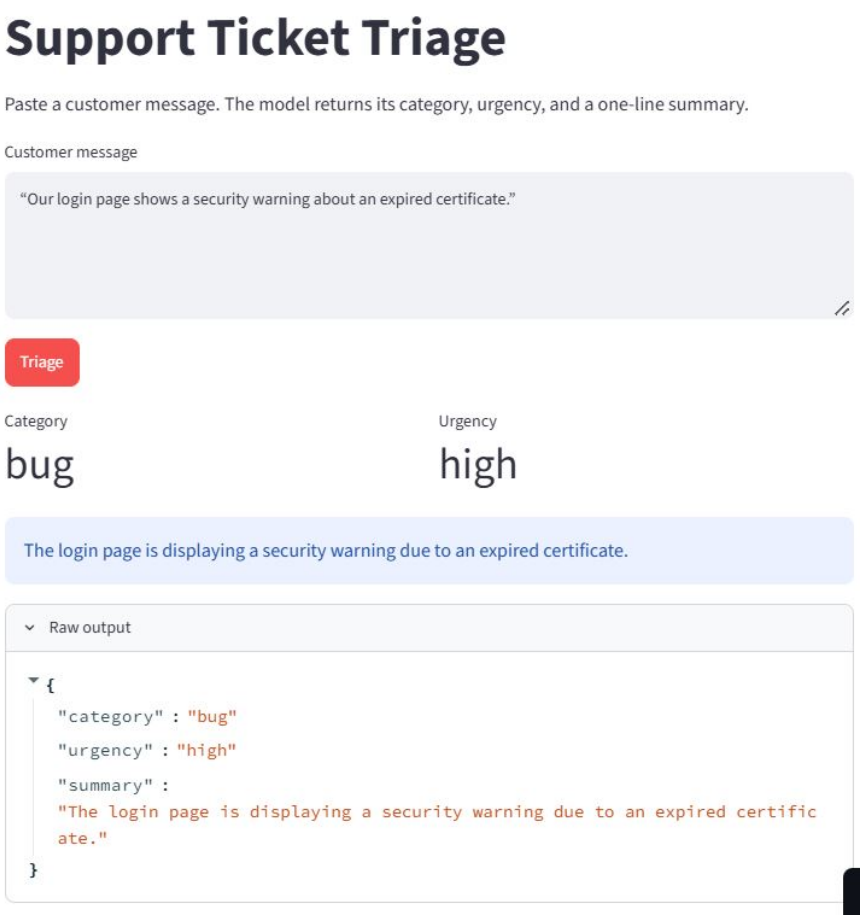
4.2 Live tests from the deployed app

The live app was tested with messages written the way a customer might send them. Three of the results are shown below. Together they cover the three things a triage tool must do: recognize a high-priority issue, handle a message that does not belong to support, and separate category from urgency.

Test A: security issue, expected high priority

Message: *"Our login page shows a security warning about an expired certificate."*

Result: category **bug**, urgency **high**. Summary: "The login page is displaying a security warning due to an expired certificate."

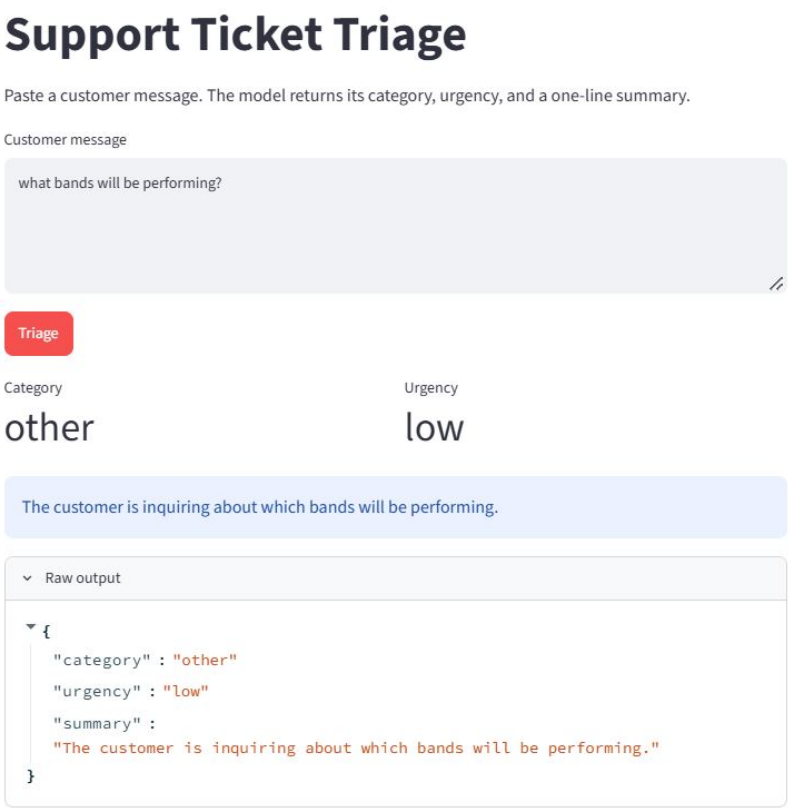


This is the behavior the urgency guide asks for. A security warning on a login page affects every user and can stop people from signing in, so high is the correct priority. The summary restates the problem in neutral terms that a support lead could act on without reading the original message. The category is bug, which is reasonable because the page is malfunctioning.

Test B: off-topic message, expected other and low

Message: *"what bands will be performing?"*

Result: category **other**, urgency **low**. Summary: "The customer is inquiring about which bands will be performing."

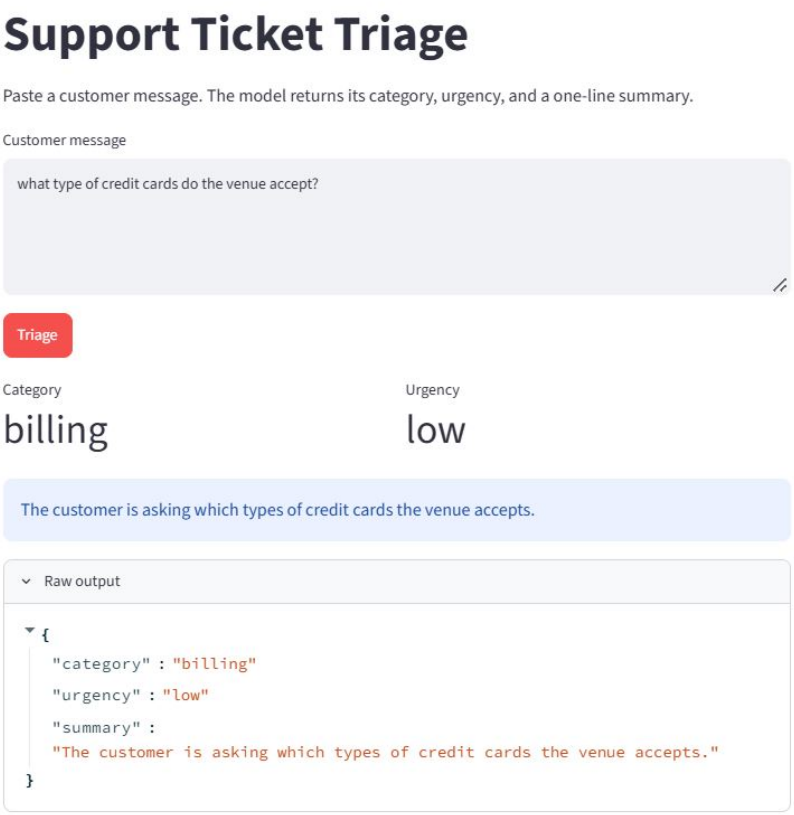


This shows the tool is not forced into a wrong category. A question about a music lineup does not fit bug, billing, or feature request, so the model uses the catch-all category and marks it low because nothing is blocked. For a real inbox, this is useful: unrelated mail can be sorted into a low-priority bucket instead of contaminating the bug queue. A natural extension is an explicit spam or out-of-scope label, which would be a reasonable next step.

Test C: billing question, expected billing and low

Message: *"what type of credit cards do the venue accept?"*

Result: category **billing**, urgency **low**. Summary: "The customer is asking which types of credit cards the venue accepts."



This test separates two ideas the system must keep apart. The topic is payments, so billing is the correct category. But the customer is asking for information and reports no problem, so urgency is low. A triage system that treated every payment-related message as urgent would overload the billing team. Getting both fields right on the same message is the main reason the output has two separate fields rather than a single priority score.

Across all tests, the summaries are accurate, the output follows the JSON structure, and the labels follow the written rules. One earlier test with a clearly blocking message, "The app crashes every time I upload a photo. I cannot work today," also returned bug and high. The tests confirm the deployment, the secrets configuration, and the model connection work from end to end.

5. What makes this project stand out

- **It runs as a product, not a notebook.** A public URL, a secured key, and a repository anyone can read.
- **It measures itself.** Accuracy is computed against labeled data, and the misses are analyzed instead of hidden.
- **It is honest about status.** Partial results are labeled as partial, and quota limits are explained rather than presented as bugs.

- **It makes engineering decisions visible.** Structured output, validation, deterministic settings, and centralized configuration each have a stated reason.
- **It generalizes.** The same pipeline handles other startup classification problems by changing the category lists and the prompt.

6. Limitations and next steps

- Only 20 hand-labeled examples. A larger, real-world dataset would give more reliable accuracy estimates.
- Full evaluation pending quota reset. The README will be updated with the complete results.
- No integration with ticketing systems such as Zendesk, Intercom, or Jira.
- No human-review queue for low-confidence or high-urgency results.
- No explicit confidence score. Adding one, for example by asking the model for a rating and calibrating it against the labeled set, would support the human-review step.
- Not yet tested on attachments, multiple languages, or abusive input.
- Next experiment: compare two models or two prompt versions on the same test set, and report accuracy, latency, and cost per ticket.

7. Links

- Source code: https://github.com/shay2222/support-triage
- Live app: https://support-triage-qbzvsmzvf8smjdbuvykeqc.streamlit.app/

Note: the live app uses a free-tier API key with daily limits. If it shows a quota error, that is the limit, not a bug.

8. Summary

Support Ticket Triage is a small, complete example of putting a large language model into a real workflow. It moves from a clear problem, through prompt and schema design, validated output, and measured evaluation, to a deployed interface. Its results are reported with their limits, and its next steps are specific. The same approach applies to lead routing, bug intake, feedback analysis, and finance operations, which are problems most early-stage companies face.